Distribution-Robust Vector Search: Calibrated Routing, Coverage, and Precision-Adaptive Scanning for IVF-PQ

Thomas Dybdahl Ahle
Normal Computing
thomas@normalcomputing.com

Abstract

Scalable approximate nearest-neighbor (ANN) search routes a query to a few partition cells, scans quantized codes inside them, and re-ranks a shortlist. We present a single IVF-PQ engine that is *distribution-robust*: on *out-of-distribution* (OOD) workloads — cross-modal text→image retrieval under inner product — it outperforms ScaNN at 1M and 10M under a same-hardware, tight-pairwise protocol (ScaNN at its official configuration at 10M and at its swept speed-optimal configuration at 1M), indexes 100M where no ScaNN configuration builds within a 2h gate, and transfers to the streaming track; on *in-distribution* high-dimensional workloads (Cohere-768, cosine; Wikipedia Cohere-v3, d=1024) it beats HNSW in HNSW's home regime at every measured operating point — graph-free at 1M (1.5–2.4× QPS through HNSW's recall ceiling, which it exceeds), and ∼1.2–1.9× at 10M and 1.5–3.3× at 35M after a minutes-scale internal graph build. The design rests on two *bottleneck localizations*, each supported by negative results reported as first-class evidence. First, OOD recall is capped by *routing coverage* — which cells are probed and which points are reachable — not by scan throughput or code fidelity: a battery of scan- and code-side improvements (wider SIMD, higher-resolution and rotated codes, RaBitQ, ADC routing) leaves OOD recall/QPS essentially unmoved, while each routing- and coverage-directed technique yields a measured gain: a one-parameter routing recalibration (γ) grounded in a cap-versus-ball selectivity argument; k-NN-graph coverage augmentation; joint cross-level partition optimization; multi-assignment; and a corollary that *reverses* standard practice — once graph coverage is present, coarser cells win. Second, *scan precision must scale with code length*: the int8-saturating fast-scan caps each subspace lookup table at ∼4 bits, and the resulting noise grows as $\sqrt{m}$ with subspace count — harmless at m=100 (d=200, wide OOD margins), fatal at m=384 (d=768, small cosine margins), where it collapses in-pool ordering to 0.07; an int16 scan of the *same codes* restores ordering to the pool ceiling (0.87), so the engine auto-selects scan precision by m. The coverage graph is *self-built*: an internal NN-descent pass reaches 0.90 overlap@16 at 10M in minutes — less than HNSW's own index build time — because local join never routes, sidestepping the d=768 coverage cap that binds routed self-search. All comparisons are same-hardware sequential pairs, and the measurement methodology is treated as a contribution.

1 Introduction

Approximate nearest-neighbor (ANN) search is the retrieval primitive under retrieval-augmented generation, semantic search, and recommendation. At scale the dominant family is inverted-file with quantization (IVF-PQ): partition the database into C cells, at query time *route* to the $p \ll C$ most promising cells, *scan* compressed codes within them, and exactly *re-rank* a shortlist. A large literature has driven each stage — anisotropic quantization and SIMD fast-scan for the scan, product and residual codes for memory, graph indices as an alternative to IVF — overwhelmingly evaluated where queries and database are drawn from the *same* distribution.

We study *distribution robustness*: one engine, measured in two regimes that break different stages of the pipeline. The first is *out-of-distribution* (OOD) search, where the query distribution differs from the database. This is not a corner case: cross-modal retrieval embeds a text query and searches image vectors under inner product, and the big-ANN [9] “text2image” track (image base, text queries, d=200) is exactly this setting. It is where the strongest systems separate, and — we argue — where the conventional intuition about what to optimize is wrong. The second is the *in-distribution, high-dimensional* regime (Cohere-768 embeddings under cosine), the bread-and-butter workload of graph indices like HNSW [3] — where IVF-PQ engines are conventionally expected to trail. We show the gap is neither routing nor architecture but arithmetic precision in the scan (§11): a single scan-precision policy is worth 2× against HNSW at d=768, and with it the same engine wins outright (Figure 13).

Our organizing claim is a *bottleneck localization*: on OOD workloads the binding constraint is *routing coverage* — whether the true neighbor’s cell is among the p probed and its members reachable — not scan throughput or code fidelity. We support this from both sides. A battery of scan- and code-side improvements (wider SIMD, higher-resolution and rotated codes, RaBitQ [8], ADC routing [10]) leaves OOD recall and QPS essentially unmoved (§13); meanwhile a small set of routing- and coverage-directed techniques each yields a measured gain (§6–§10). We therefore present the work as a measured collection — every technique reported with the effect it produces in isolation, and the negatives kept as first-class evidence for the localization. Composed and evaluated under a same-hardware, tight-pairwise protocol against ScaNN [1] at its best configuration, the result attains higher QPS at matched recall@10 at 1M and 10M (and, where ScaNN cannot build within a 2h gate, remains the only method to index 100M), and transfers to the streaming track.

Contributions (each measured in isolation).

- **A bottleneck localization** (§5): scan- and code-side levers do not move OOD; routing/coverage does. The negatives (§13) are the evidence.
- γ **routing recalibration** (§6), grounded in a cap-vs-ball selectivity argument (§4). *Effect:* $\approx$ *half the probes at matched OOD recall, zero query-time cost.*
- k**-NN-graph coverage** (§7). *Effect: composes with* γ*; at 1M moves the ScaNN ratio* 1.18 → 0.98.
- **The coarse-cell corollary** (§8). *Effect: with graph coverage and tree-EM present, coarser cells win; at 10M* 0.92 → 0.85.
- **Joint cross-level partition optimization (tree-EM)** (§9). *Effect:* +0.6–1.0 *recall@10 points at 10M, QPS-neutral.*
- **Multi-assignment (SOAR [2])** (§10). *Effect: raises the routing recall ceiling to baseline parity.*
- **Scan precision as a function of code length** (§11): the int8-saturating fast-scan’s per-subspace ∼4-bit LUT cap injects noise growing as $\sqrt{m}$; at m=384 (in-distribution d=768 cosine) it collapses in-pool ordering to 0.07 while the *same codes* under an int16 scan rank to the pool ceiling (0.87). *Effect: the precision policy alone is worth* 2× *against HNSW at* d=768 *— the difference between trailing and winning. With precision auto-selected by* m *(plus an AVX-512 int16 kernel), the engine beats HNSW graph-free at 1M and, with graph coverage,* ∼1.2–1.9× *at 10M — without abandoning the IVF-PQ cascade.*
- **Self-built coverage graph via internal NN-descent** (§7): local join never routes, so the d=768 coverage cap that binds routed self-search does not apply. *Effect: at 10M, a* 0.864*-overlap graph in* 24 *min from random init, or* 0.904 *in* ∼43 *min total (*35*-min routed self-kNN pass +* 8 *min descent) — either under HNSW’s own* 48*-min build — and* 0.965 *at 35M in* 9.5 *min; the 10M/35M wins are fully self-contained.*
- **Systems levers for streaming** (§12): batch-parallel insert (4.5×), low-live adaptive p (*worst step* 0.74 → 0.94), and parallel re-rank dispatch (7.6× *at 8 threads*).
- **A same-hardware evaluation methodology** (§15) and an honest scope of what the ratios do and do not claim.

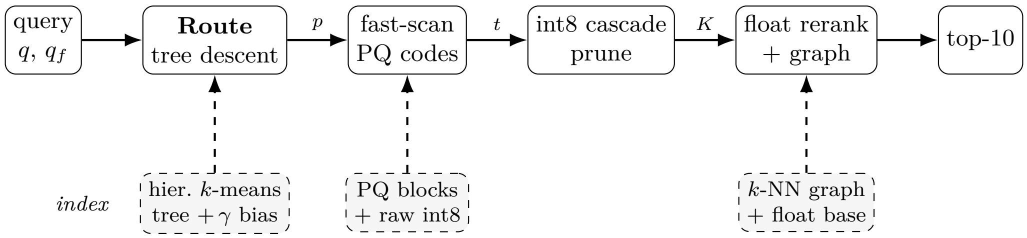


Figure 1: Query pipeline (top) and the offline-built index it consumes (bottom, dashed). Edge labels are the shrinking candidate set: p probed leaves → top-t pool → K cascade survivors → top-10. γ calibration, the k-NN graph union, and the coarse-cell tree geometry are this work's contributions (Table 1).

2 Background: Standard Building Blocks

Our system is assembled from well-established components; we summarize them here so the contributions in §3–§10 can be stated as deltas against known practice.

Inverted file (IVF). Partition the database into C cells (the Voronoi regions of C centroids). At query time, *route* the query to its $p \ll C$ nearest centroids and scan only those cells. End-to-end recall is capped by whether the true neighbor's cell is among the p probed — the routing question that this paper argues is the OOD bottleneck.

Hierarchical routing. Scoring all C centroids per query is $O(Cd)$, which dominates at scale (fine C is needed to keep cells small). A *tree* of centroids — coarse k-means over C_0 centroids, then fine k-means within each — reduces routing to $O(L\,C^{1/L})$ for an L-level tree (FAISS [4] uses an HNSW graph over the centroids for the same purpose). We use hierarchical k-means.

Product quantization (PQ). Split each vector's d dimensions into m subspaces and quantize each subspace to a small codebook (16 codewords for 4-bit PQ). Approximate distances are computed by lookup-and-add over per-subspace tables (ADC). With 4-bit codes the table lookup is a register shuffle (`vpshufb`), enabling the SIMD "fast-scan" kernels (Quick-ADC, ScaNN, FAISS FastScan). *Anisotropic* / score-aware quantization (ScaNN) weights the quantization loss to preserve inner-product ranking rather than reconstruction.

Exact re-ranking. The quantized scan is used only to select a shortlist; exact distances (int8 or float, read for the few shortlisted vectors) re-order it. The shortlist depth t trades recall for cost.

Multi-assignment. Replicating each point into its top-a_0 cells (a space/time tradeoff) raises the chance a query's neighbor lands in a probed cell.

The OOD workload. The big-ANN OOD track (text2image) pairs *image*-embedding base vectors (d=200) with *text*-embedding queries under inner product, with a float ground truth. Base norms are concentrated but not constant (≈ 0.81–0.99); queries are unit-norm. This distribution gap between base and queries is what breaks L_2-trained routing (§6).

3 System Overview

Figure 1 shows the pipeline; we describe the build and query paths, then delineate standard vs. contributed components (Table 1).

Algorithm 1: Build

Input: float base $X_f \in \mathbb{R}^{n\times d}$; tree sizes C_0, K_f; multi-assign a_0; calibration γ; EM rounds R; graph degree k

Output: index $(\{Q_0\}, \{c_j\}, b, \mathrm{PQ\ blocks}, \mathrm{raw}, G)$

$\sigma \leftarrow 127/\max_i \|X_f[i]\|_\infty$; $X \leftarrow \mathrm{int8}(\sigma X_f)$; `// global-scale quantize`

$\{Q_0\} \leftarrow k$-means(X, C_0); $\{c_j\} \leftarrow$ fine k-means within each coarse cell to K_f leaves;

for *R rounds* `// tree-EM (§9); optional` **do**

    E: assign each sample to its nearest leaf via `Route`$(\cdot, 1)$;

    M: recompute $\{Q_0\}, \{c_j\}$ from the assignments;

foreach *point* x_i `// SOAR multi-assign (§10)` **do**

    assign x_i to its top-a_0 leaves; spill covers the residual of the 1st;

per leaf: encode members as 4-bit anisotropic PQ, pack into 16-row `vpshufb` blocks; store raw int8 in leaf order;

$b_j \leftarrow (\gamma - 1)\|c_j\|^2$ for all j ; `// routing bias`

$G \leftarrow$ base-side k-NN graph (self-search; §7);

return index $(\{Q_0\}, \{c_j\}, b, \mathrm{PQ\ blocks}, \mathrm{raw}, G)$;

Build path.

1. **Quantize.** Float base $\to$ int8 via a single global scale ($127/\max|x|$); the int8 base is the native scan/route representation.
2. **Train the routing tree.** Hierarchical k-means: C_0 coarse centroids, then fine centroids within each coarse cell to K_f leaves total (2-level; generalizes to L levels).
3. **Joint refinement (tree-EM, §9).** Optionally re-optimize all levels jointly (E: reassign sample points to their nearest leaf via the beam descent; M: recompute every level's centroids).
4. **Multi-assign (SOAR, §10).** Assign each point to a_0 leaves with spill that covers the residual direction of the first.
5. **Encode.** 4-bit anisotropic PQ ($m{=}d/2$ subspaces); pack codes into cell-contiguous 16-row blocks laid out for the `vpshufb` fast-scan. Store the raw int8 vectors in cell order too (cache-warm re-rank gathers).
6. **Calibrate & augment.** Precompute the per-cell γ routing bias (§6); build the base-side k-NN graph sidecar (§7).

Query path.

1. **Route (tree descent).** Score the C_0 coarse centroids (int8, VNNI kernel), keep the top-b_0; expand to their fine children, score those with the γ bias added, and return the top-p leaves.
2. **Scan.** Fast-scan the 4-bit PQ codes of the p probed cells (`vpshufb` ADC with int16 accumulation for $\sim$12-bit selection resolution; optional AVX-512 64-wide), collecting a bounded top-t pool.
3. **Cascade re-rank.** Deduplicate the pool by original id (needed under multi-assignment); exact-rescore the t survivors in int8 (VNNI dot); prune to the top-K ($K{=}16$).
4. **Float re-rank + graph union.** Compute exact float inner products for the K survivors (reading the original float vectors) unioned with the k-NN graph neighbors of the top-M; return the top-10.

Algorithms 1–3 state the pipeline precisely. All routing scores use the calibrated form $s_\gamma(q, c) = \gamma\|c\|^2 - 2\, q\cdot c$ (§6); the constant $\|q\|^2$ is dropped, and smaller s_γ means a better cell. Leaf centroids are $\{c_j\}_{j=1}^{K_f}$ with precomputed bias $b_j = (\gamma - 1)\|c_j\|^2$ so that $s_\gamma(q, c_j) = \|c_j\|^2 - 2q\cdot c_j + b_j$ is one dot product plus an additive constant.

Algorithm 2: Route (tree descent) – returns the p best leaves

Procedure `Route(`q, p`)`**:**
  $S_0 \leftarrow$ top-b_0 coarse cells minimizing $s_\gamma(q, Q_0)$; `// int8/VNNI kernel`
  $\mathcal{L} \leftarrow$ fine children of S_0;
  score each $c_j \in \mathcal{L}$ by $\|c_j\|^2 - 2q \cdot c_j + b_j$;
  return the p leaves of $\mathcal{L}$ with smallest score;

Algorithm 3: Query

Input: int8 query q, float query q_f; probes p; pool depth t; prune K; graph fan-in M
$\mathcal{C} \leftarrow$ `Route(`q, p`)`;
$\mathcal{P} \leftarrow \emptyset$; `// bounded top-`t` pool`
foreach *cell* $\in \mathcal{C}$ **do**
  fast-scan its 4-bit PQ codes (`vpshufb` ADC, int16 accum) $\rightarrow$ approx. scores;
  keep the t best in $\mathcal{P}$ (threshold-bounded collect);
$\mathcal{P} \leftarrow$ dedup $\mathcal{P}$ by original id ; `// needed under `$a_0 > 1$
$\mathcal{K} \leftarrow$ top-K of $\mathcal{P}$ by exact int8 IP (VNNI dot) ; `// cascade prune`
$\mathcal{R} \leftarrow \mathcal{K} \cup \bigcup_{r \in \text{top-}M(\mathcal{K})} G[r]$; `// `k`-NN-graph coverage union`
return top-10 of $\mathcal{R}$ by exact float IP $\langle q_f, x_r \rangle$;

4 Theoretical Grounding (Scoped)

Under concentration, additive ball filters become non-selective — in high dimension almost everything falls within $R+r$ — whereas angular (spherical-cap) filters remain selective, and k-means on centered data realizes exactly such a data-dependent angular partition. This framing motivates γ: the routing score $\gamma\|c\|^2 - 2q \cdot c$ interpolates from an additive ball (γ=1, L_2) to a pure inner-product cap ($\gamma \rightarrow 0$), so the geometry predicts that $\gamma < 1$ is needed precisely in the concentrated OOD regime — which our experiments confirm. We build on two principles from this line of work: *route-precise, rank-cheap*, and *multi-assignment as a space/time tradeoff*. We do *not* claim the recursive-tree polylog-loss result, which does not pay off at the scales evaluated here (up to 100M).

5 Localizing the OOD Bottleneck

We separate two ceilings (Figure 2). The *pool-recall ceiling* is the recall obtainable by exactly re-ranking the *entire* probed pool — it depends only on routing (which cells, hence which points, are reachable), not on the scan's approximation. The *scan-limited recall* is what the actual (quantized) scan plus shortlist delivers. If the two coincide, the scan is already extracting everything the routed pool contains and there is no scan headroom; recall is then a pure function of routing coverage.

On OOD text2image this is what we observe: at a fixed probe budget the scan-limited recall sits at the pool-recall ceiling, and that ceiling — not the scan — is what moves when we change routing. Two consequences follow, and the rest of the paper is their systematic exploration. First, techniques that improve *coverage* (route calibration, graph augmentation, joint partition optimization, multi-assignment) should raise recall; §6–§10 confirm each does, with a measured effect. Second, techniques that only improve the *scan or the code* (wider SIMD, higher-resolution or rotated codes, alternative quantizers, ADC routing) should not help, because there is no scan headroom to recover; §13 confirms each does not. The negatives are thus not incidental — they are the control that localizes the bottleneck to routing coverage.

Table 1: Components of the pipeline: standard building blocks vs. this work.

Component	Basis	Our delta
int8 IVF partition	standard	—
Hierarchical k-means tree	standard (FAISS-style)	fast int8/VNNI descent kernel
4-bit anisotropic PQ	ScaNN / FastScan family	int16-resolution + 32/64-wide fast-scan
Multi-assignment (SOAR)	ScaNN-derived	dedup-before-cap correctness fix
int8→float re-rank cascade	folklore	the specific K=16 composition
γ routing calibration	**ours**	the OOD miscalibration fix
k-NN-graph coverage union	**ours**	composition with γ + float re-rank
Coarse-cell geometry knee	**ours**	reverses "finer is better"
Tree-EM joint optimization	**ours**	cross-level partition objective
Batch-insert / adaptive-p (streaming)	**ours**	§12

6 γ: One-Parameter Routing Recalibration

An IVF router ranks cells by a score over the query q and centroid c. For L_2 k-means the score is $\|q-c\|^2 = \|q\|^2 + \|c\|^2 - 2\,q\cdot c$; dropping the query-constant term leaves $\|c\|^2 - 2\,q\cdot c$. We write the family

$$s_\gamma(q,c) \;=\; \gamma\|c\|^2 - 2\,q\cdot c, \tag{1}$$

where γ=1 recovers L_2 routing and $\gamma\to 0$ recovers pure inner-product (angular) routing. Under distribution shift these disagree in a specific way: the $\|c\|^2$ penalty demotes high-norm centroids, but for OOD queries the inner-product-relevant neighbors concentrate precisely near those high-norm cells, so γ=1 systematically misroutes them. A single global $\gamma<1$ rebalances the two terms. It is grounded in the cap-vs-ball argument of §4, illustrated in Figure 4; Figure 3 shows the measured effect: γ slides routing along the additive-ball (γ=1, non-selective under concentration) to spherical-cap ($\gamma\to 0$, selective) axis.

Crucially the fix is *free at query time*: s_γ differs from the L_2 score only by the per-cell constant $(\gamma-1)\|c\|^2$, which we fold into a precomputed integer bias added during cell scoring (no extra query work, no index-format change). We fit γ once per dataset; $\gamma\approx 0.5$ is optimal on text2image at both 1M and 10M, and the recall is flat across $\gamma \in \{0.4, \ldots, 0.6\}$ near the optimum.

Effect. Reaches matched OOD recall at ≈half the probes (p: 54→29 at 1M; ≈2.5× probe cut at 10M). Zero query-time cost.

7 k-NN-Graph Coverage Augmentation

Routing to p cells can still miss a true neighbor whose cell went unprobed. We recover such misses with a graph (Figure 5), without probing more cells. Offline we build a base-side inner-product k-NN graph (a per-point adjacency list; constructed by ScaNN self-search, by the engine querying itself, or — where routed self-search is coverage-capped — by the engine's internal NN-descent pass, below). The graph is a method-agnostic offline artifact, so the query-time comparison is unaffected by which builder is used, and we verified this empirically: on OOD text2image-10M at the champion config, swapping the ScaNN-built graph for the engine's own self-built one changes recall by ≤0.1pt and QPS within noise. Where the source could matter for a claim we use the self-built graph outright — e.g. the 35M in-distribution result (Table 6) uses the engine's *own* NN-descent graph, which matches an HNSW-built one (0.965 vs 0.970 overlap@16) — so no comparison borrows anything from the opponent. At query time, after the scan yields a shortlist, we take the top-M shortlist members and *union their graph neighbors into the exact re-rank set.* The intuition is transitive: a shortlisted point near the query is itself near the true neighbor, so the true neighbor tends to appear among that point's stored nearest neighbors even when its own cell was not probed. This adds coverage that is *independent of the partition* — the property that drives the coarse-cell corollary (§8). It composes with γ: the two attack different misses (calibration fixes which cells are ranked highly; the graph reaches beyond the probed cells), overlapping only partially. The union is deduplicated against the scanned

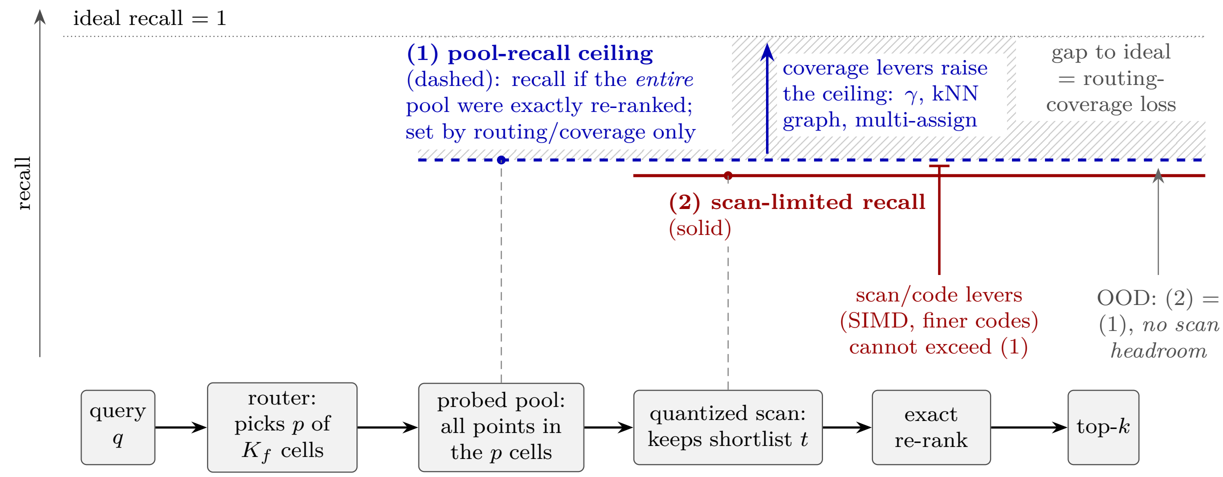


Figure 2: Localizing the OOD bottleneck in the IVF–PQ pipeline (conceptual diagram; recall levels are schematic, and the two bars are drawn with a small vertical offset only for legibility). A query is routed to p of K_f cells; the probed pool fixes the *pool-recall ceiling*—the recall attained if the entire pool were exactly re-ranked—which depends only on routing coverage, while the quantized scan's shortlist t yields the *scan-limited recall*. On out-of-distribution queries the two levels coincide: the scan already saturates the pool ceiling, so the residual gap to ideal recall is pure routing-coverage loss (hatched). Coverage levers (γ-routing, kNN-graph augmentation, multi-assignment) therefore raise the ceiling, whereas scan and code refinements (SIMD kernels, finer codes) cannot lift recall above it.

pool (already-scanned neighbors are not re-scored) and the adjacency reads are software-prefetched, so the added cost is a small, bounded re-rank increment.

Effect. At 1M, composed with γ, the ScaNN ratio drops 1.18 → 0.98; at 10M it lets t_{surv} halve at matched recall. Graph parameters scale with n: fan-in M and pool depth grow from $M{\approx}25$ at 1M to $M{=}64$ at 100M as distractors multiply.

Repair depth (and where it helps). The union above is one hop. Iterating it — expand the current best-M members' neighbors, re-score, reselect, for R rounds — is a *batched* graph walk (beam width M; the whole cohort is SIMD-rescored and a single exact re-rank closes it, so no per-query priority queue). It has a clean cost/quality model: each hop closes a geometric fraction (≈56%) of the still-repairable miss mass, while p and t_{surv} raise a per-probe reachability *ceiling* that hops cannot exceed (fit to $R^2{=}0.99$ on a 20-point grid, so the two knobs tune independently, no black-box search). Crucially, extra hops are a matched-recall win *only above a* ≈0.91–0.92 *crossover*: at the leaderboard's recall@10≥0.90 gate the single-hop point is ≈6% faster (a second hop reaches 0.90 only by *overshooting* recall at a QPS cost), whereas for high-recall targets (0.94–0.96) a second or third hop reaches them at far lower p than one hop. We therefore keep hop count a knob (default 1), not a default — a high-recall lever, not a gate-level one. A best-first frontier (expand the globally-best unexpanded node) adds +0.3–0.8 pt *in-distribution* but is neutral OOD: base-metric edges do not align with off-manifold queries, so smarter reselection finds no better seeds. Reporting this required comparing at the recall *gate*, not fixed p — a fixed-p "+1.7 pt" reads as a win but is not one at the gate.

Self-building the graph: internal NN-descent. How the graph is *built* matters for self-containment (Figure 6). The engine's *routed* self-search cannot build a good graph at 10M/d=768: routing is coverage-capped (§18), and no practical probe depth fixes it (0.63 overlap@16 even probing 6% of cells). NN-descent [7] sidesteps routing entirely: starting from a random (or cheaply routed) adjacency, each round joins every point against its neighbors-of-neighbors — forward and (capped) reverse edges — and keeps the top-k; convergence relies on transitivity of proximity, not on any partition. Our implementation is a race-free *pull* variant (each point updates only its own row; int8 VNNI dots; per-thread stamped-hash dedup) with two fixes that matter at inner-product hubness: the capped reverse lists are *reservoir-resampled each round* (a first-come

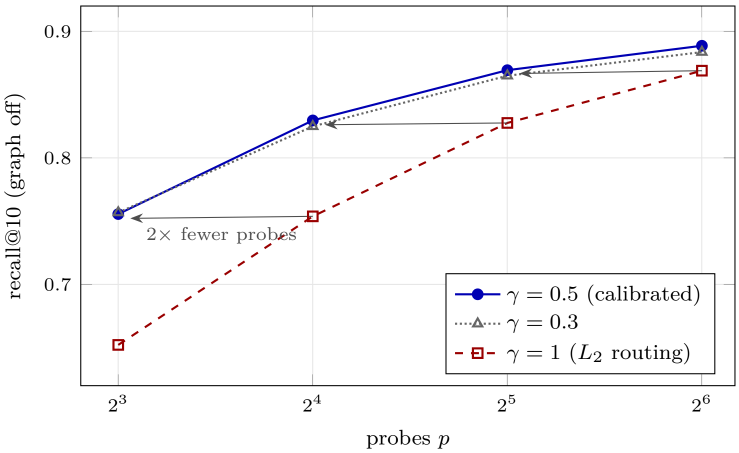


Figure 3: The γ recalibration effect on OOD routing (text2image-10M, graph off to isolate routing; recall is deterministic). At every operating point, γ=0.5 matches the recall of L_2 routing (γ=1) at half the probes — 0.7556 at p=8 vs 0.7538 at p=16, 0.8296 at 16 vs 0.8276 at 32, 0.8693 at 32 vs 0.8689 at 64 — a uniform horizontal shift of the routing curve at zero query-time cost. γ=0.3 overshoots slightly, bracketing the optimum.

cap freezes hub in-neighbors and permanently misses their pairings), and inserted edges stay "new" for two rounds (one round of gating loses join chances at saturated hubs).

Effect. Cohere 10M (d=768): random init reaches 0.864 overlap@16 in 24 min (16 threads); warm-starting from a routed self-kNN pass (itself 35 min) reaches **0.904** with 8 min more of descent — either path costs less than HNSW's own 10M index *build* (48 min, same box), and is query-time equal-or-better than the multi-hour external near-exact graph (recall 0.9871 vs 0.9869 at matched probes; 0.9904 vs 0.9901 at the top point). At 1M: 0.895 in ∼1 min; at 35M/d=1024 (Wikipedia): 0.874 → 0.965 in 9.5 min, matching an HNSW-built graph's 0.970 and adding +0.3–0.45pt recall at the same QPS. On OOD text2image-10M (d=200), where routed self-search is already near-exact (0.992 overlap), swapping in the NN-descent graph (0.970) leaves champion recall/QPS unchanged — graph quality is saturated at both ends. The 10M and 35M wins are therefore fully self-contained.

8 Routing Cost and the Coarse-Cell Corollary

8.1 A routing cost model

Consider an L-level tree over K_f leaves with cell counts $C_0 < C_1 < \cdots < C_{L-1} < C_L = K_f$ and per-level beam widths $b_0, \ldots, b_{L-2}$ (the number of cells expanded when descending from level i to level i+1). Routing a query costs, in centroid distance evaluations,

$$\text{route}(q) \;=\; C_0 \;+\; \sum_{i=1}^{L-1} b_{i-1}\,\frac{C_i}{C_{i-1}}, \tag{2}$$

since the coarse level scores all C_0 centroids and level i expands the b_{i-1} best parents from level $i-1$, each contributing C_i/C_{i-1} children. The scan then reads $\text{scan}(q) = p \cdot (n/K_f) \cdot a_0$ candidates (p probed leaves × points/leaf × multi-assignment). For a 2-level tree (2) is $C_0 + b_0 K_f / C_0$, minimized at $C_0 = \sqrt{b_0 K_f}$ to give $2\sqrt{b_0 K_f} = O(\sqrt{K_f})$; with L levels and balanced branching this falls to $O(L\,K_f^{1/L})$. The two costs pull oppositely in K_f: finer leaves shrink the scan (n/K_f) but grow the routing. The beams b_i are the "routing aggression" knob — wider beams retain the true cell with higher probability at linear cost — and OOD queries, which route poorly, need wider beams (Table 7).

Table 7 gives geometry and routing cost by scale. Two levels suffice through 10M; at 100M a 2-level tree at its optimal $C_0 = \sqrt{b_0 K_f} \approx 16{,}400$ would cost ≈33,000 evals, whereas the 3-level tree roughly halves this

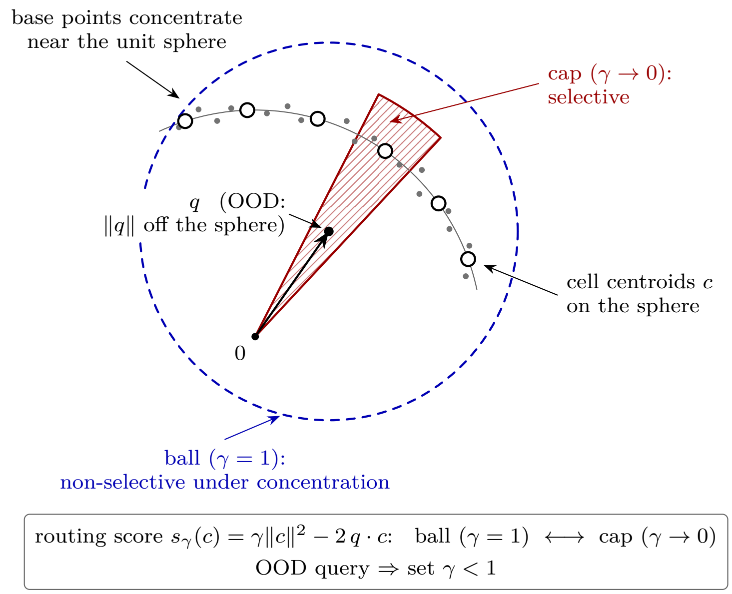


Figure 4: Why routing needs recalibration off-distribution (conceptual 2-D sketch, not data). Base points (small dots) and cell centroids c (open circles) concentrate near the unit sphere, while an out-of-distribution query q lies off the sphere. An additive L_2 ball around q ($\gamma = 1$, dashed) contains nearly all centroids once distances concentrate, so it cannot discriminate cells; the angular cap induced by the inner-product limit ($\gamma \to 0$, hatched) isolates the few centroids aligned with q. The routing score $s_\gamma(c) = \gamma\|c\|^2 - 2\,q \cdot c$ interpolates between the two regimes, so off-distribution queries call for $\gamma < 1$.

(14,336) — so the third level is introduced only at 100M, where routing is a large enough share of the query to matter.

8.2 Scaling laws for geometry

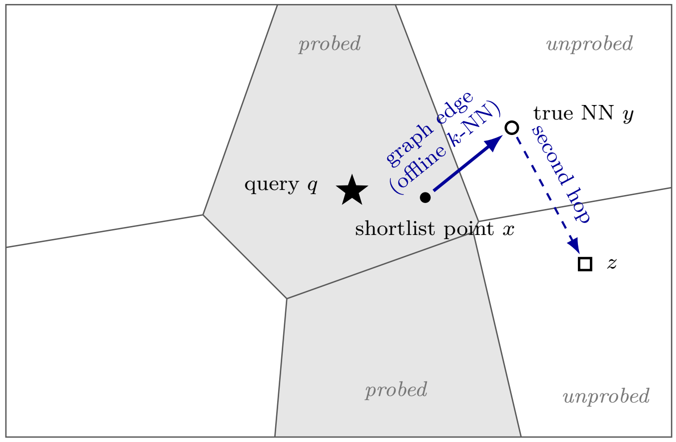
To set geometry at scale without per-n retuning, we sweep K_f at $n \in \{10^5, 3{\cdot}10^5, 10^6, 3{\cdot}10^6, 10^7\}$ (2-level, graph off), at each n finding the minimum p that reaches recall@10 ≥ 0.90 (deterministic, hence load-independent) and the resulting cost route + scan from Eq. (2). The cost-minimizing geometry follows clean power laws (Figure 7; log-log fit):

$$K_f^\star \sim n^{0.66}, \qquad \tfrac{n}{K_f^\star}\,(\text{pts/leaf}) \sim n^{0.34}, \qquad p^\star \sim n^{0.18}, \qquad C_0^\star \sim n^{0.33}.$$

The central law is pts/leaf $\sim n^{1/3}$: optimal leaves *grow* with n, because routing $O(\sqrt{K_f})$ rises while the scan $O(p\,n/K_f)$ is held down by the growing K_f — the coarse-cell corollary expressed as a scaling law. Probes grow only as $n^{1/5}$. Extrapolating, $n{=}10^9$ calls for K_f of order 10^6: the law gives $1.6{\times}10^6$ leaves ($\sim$640 pts/leaf), which we round to the power-of-two choice $K_f{=}2^{20} \approx 10^6$ ($\sim$1000 pts/leaf). These calibrate the 2-level baseline; graph coverage (§7) shifts the optimum coarser and a third routing level (cheaper, $O(K_f^{1/3})$) shifts it finer, so the 100M geometry (Table 7) is validated by a direct fine-vs-coarse K_f A/B at 100M rather than read off the 2-level law.

8.3 The corollary

The cost model (2) explains *why*. Without graph coverage, recall demands fine K_f (small leaves make the true neighbor's leaf likelier to be among the p probed), paying $O(\sqrt{K_f})$ routing. The k-NN-graph union (§7) supplies that coverage *independently of* K_f, so the tree can use *coarser* leaves — cheaper routing — at fixed recall. The optimum is a knee (Figure 8): too fine wastes routing, too coarse inflates the scan ($p \cdot n/K_f$). Empirically the knee sits near 150 points/leaf at 10M, and both finer and coarser builds lose to it.



y is unreachable by routing (its cell is unprobed), but one stored edge from x reaches it — coverage independent of the partition.

Figure 5: Mechanism of k-NN-graph coverage augmentation (conceptual sketch; the partition and points are illustrative). Coarse routing probes the $p = 2$ cells nearest to the query q (shaded), so the true neighbor y in an unprobed cell (white) is invisible to routing, no matter how the shortlist is re-ranked. A k-NN graph edge computed offline and stored at shortlist point x (solid arrow) reaches y in a single hop, and a second hop (dashed arrow) extends coverage to z in yet another unprobed cell. Recall is thus decoupled from partition quality: any neighbor within a few graph hops of the shortlist is recovered.

Effect. At 10M, with γ, graph coverage, and tree-EM held fixed, moving from the fine-cell to the 152-point-cell geometry takes the ScaNN ratio 0.92 → 0.85; routing evals/query drop ∼12,000→6,144. The same shift wins OOD at 1M: with the hybrid graph, K_f=16384 → 4096 (61 → 244 pts/leaf) is +2 recall points at +8% QPS at the gate, and it is what closes the small-scale frontier (Figure 14).

9 Joint Cross-Level Partition Optimization (Tree-EM)

Hierarchical k-means trains each level greedily top-down, so a leaf centroid is optimized for its parent's Voronoi cell, not for the query-time beam descent that actually reaches it. We instead optimize all levels jointly by EM against the search objective: the E-step assigns each sample point to the leaf its *beam descent* lands in (not its globally-nearest leaf), and the M-step recomputes every level's centroids from those assignments. Iterating co-adapts the levels so the descent lands better. The E-step is itself an ANN query against the tree's own centroids, so it runs at a shallow beam (a few probes, not the full query beam), making EM cheap; it is a build-time cost only and adds nothing at query time. It composes with γ and the graph (it improves the partition; they improve routing and coverage on top).

Effect. +0.6–1.0 recall@10 points at 10M at matched probe, QPS-neutral (build-time only).

10 Multi-Assignment (SOAR)

The remaining coverage lever is redundancy: assign each point to its top-a_0 cells rather than one, so a query reaching any of them finds the point. Naive replication puts near-duplicate copies in adjacent cells; SOAR instead chooses the extra cells to cover the *residual direction* of the first assignment, so the copies span different query directions and buy more coverage per unit storage. Realizing the gain required a correctness fix: the exact-re-rank shortlist must be deduplicated by original id *before* the survivor cap, otherwise the a_0 duplicate copies of a point crowd the fixed-size shortlist and starve distinct ids (recall then falls, and paradoxically decreases with more probes — the signature of the bug). With the fix, multi-assignment raises the routing-coverage ceiling.

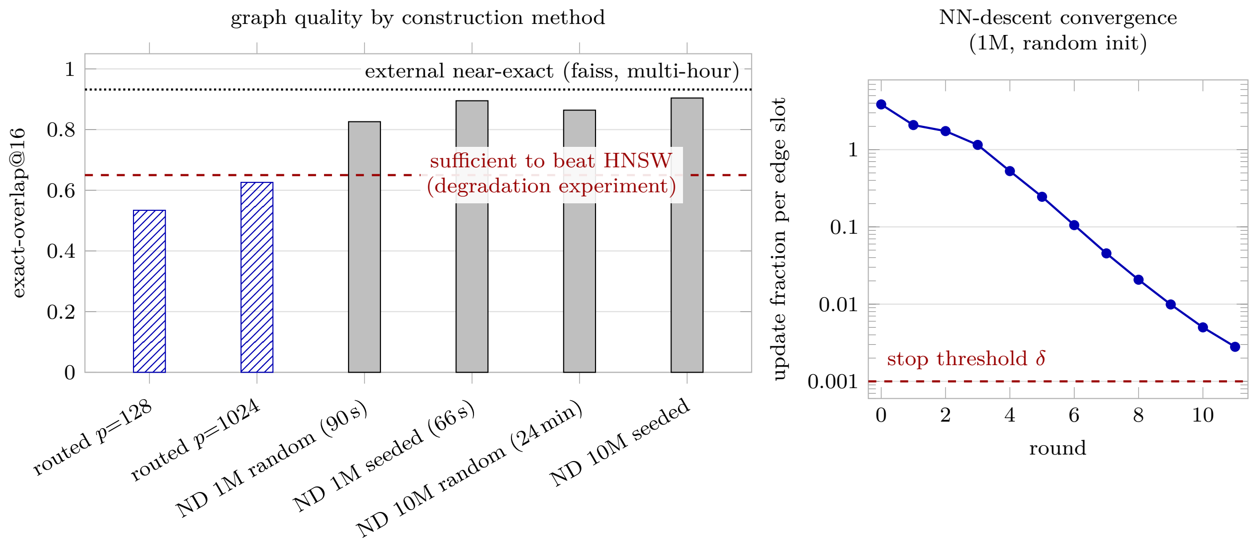


Figure 6: Self-built kNN-graph quality and NN-descent convergence on cohere-768. Left: exact-overlap@16 by construction method; routed self-search (hatched) is coverage-capped at 0.534–0.626 even with p=1024 probes (6% of cells), while NN-descent (ND, solid) reaches 0.826–0.904 at both 1M and 10M — clearing the 0.65 level that the degradation experiment shows is sufficient to beat HNSW, and approaching the 0.932 overlap of the external near-exact graph (faiss, multi-hour build). Right: per-round update fraction per edge slot for NN-descent at 1M from random init (log scale); the local join converges geometrically down to the $\delta = 0.001$ stop threshold, and because no routing runs in the loop, the coverage cap does not apply. Seeding NN-descent from the routed graph both shortens the 1M build (66 s total vs. 90 s from random init) and lifts final overlap at both scales (0.895 vs. 0.826 at 1M; 0.904 vs. 0.864 at 10M).

Effect. a_0=2 reaches OOD coverage parity with the baseline partition (Figure 9); higher a_0 extends the high-recall frontier at a storage/scan cost linear in a_0 (the a_0 factor in the scan term, §8.1).

11 Scan Precision Must Scale with Code Length

The techniques so far repair *routing and coverage*, the OOD bottleneck. The in-distribution, high-d regime breaks a different stage, and localizing it is a second instance of the paper's method: measure stages in isolation until the failing one confesses.

The symptom. On Cohere-768 (10M, cosine, unit-norm) the engine needed an exact-re-rank pool of t≈32,000 to reach recall 0.90 — 60× the OOD operating depth — and recall *decreased* with more probes. Stage-isolation pinned it: the routed pool *contains* the true neighbor (pool ceiling 0.87–0.94), and an 8-bit re-ranker recovers exactly the ceiling, but the 4-bit fast-scan's *own ordering* of the pool is catastrophic — the true neighbor almost never ranks in its top-100 (in-pool ordering recall 0.07).

The cause is arithmetic, not codes. The SIMD fast-scan accumulates lookup-table (LUT) values in *saturating int8*; to avoid saturation within a hoist group, each subspace's LUT is quantized to a ∼4-bit range (cap $\lfloor 120/\text{HOIST} \rfloor$=15). The resulting per-subspace quantization noise is independent across subspaces, so the total scan-score noise grows as $\sqrt{m}$ (Figure 10, left) while the useful margin between neighbors does not. At d=200 (m=100 subspaces, wide OOD margins) this is harmless — which is exactly why the int8 path is the right champion default there. At d=768 (m=384, small in-distribution cosine margins) the noise swamps the signal. The discriminating experiment (Figure 10, right) holds the *index and codes fixed* and changes only the scan arithmetic: int8-saturating scan orders the pool at 0.07; an int16-LUT scan orders it at 0.87 — the pool ceiling — at rank depth 100.

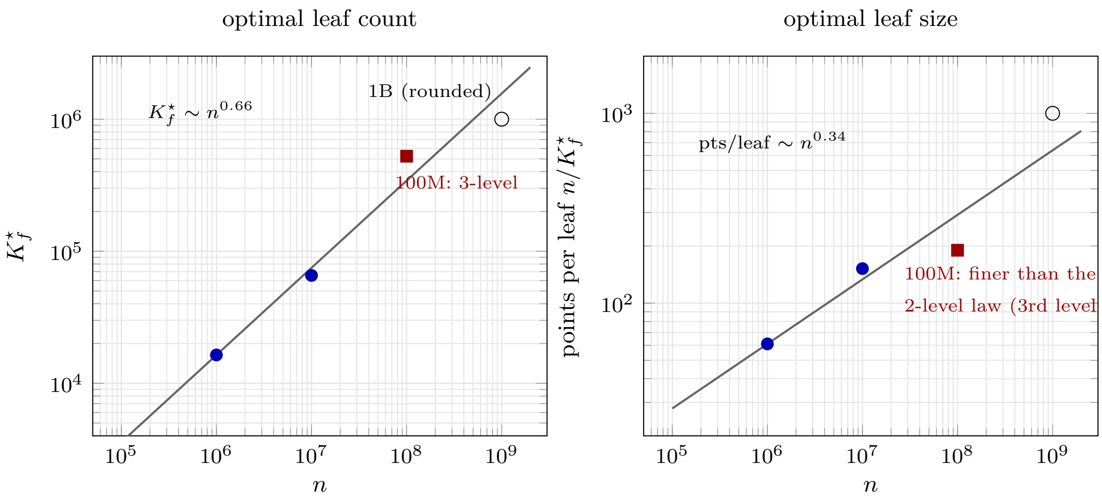


Figure 7: Scaling laws for index geometry (log-log). Lines are the power laws fitted on the 2-level, graph-off K_f sweep at $n \in \{10^5, \ldots, 10^7\}$ ($K_f^\star \sim n^{0.66}$, hence pts/leaf $\sim n^{0.34}$: optimal leaves *grow* with n — the coarse-cell corollary as a scaling law); filled circles are the built 1M/10M geometries. The 100M point (square) deliberately deviates finer: its third routing level makes routing cheaper ($O(K_f^{1/3})$), shifting the optimum, so it is validated by direct A/B rather than read off the 2-level law. The open circle marks the projected 1B geometry: the law's raw extrapolation (1.6×10^6 leaves, $\sim$640 pts/leaf) rounded to the practical power-of-two choice $K_f{=}2^{20}\approx10^6$ ($\sim$1000 pts/leaf), so the marker sits slightly coarser than the drawn line. Exact cell counts, beams, and per-query route evaluations are in Table 7.

The fix is a policy, not a rewrite. Scan precision is selected by subspace count: $m \leq 128$ keeps the int8 fast-scan (bit-identical to the OOD champion path), $m > 128$ uses int16 LUTs. Two systems consequences follow. First, the survivor cut becomes trustworthy, so the re-rank pool collapses from 32,000 back to a few hundred. Second, the int16 kernel is worth engineering: a 32-wide AVX-512 (`vpermw`) int16 pair-scan — with its dispatch hoisted out of the per-block hot loop, where a per-call environment lookup would cost $\sim$25% of the scan — is compute-bound at $m{=}384$ (sorted-vs-scattered block order = 1.00×), recall-bit-identical, and +31% end-to-end. Notably this *reverses* the wider-SIMD negative of §13: at $m{=}100$ the scan is memory-bound and AVX-512 buys $\leq 7\%$; at $m{=}384$ it is LUT-compute-bound and the same width is decisive. Precision and width are functions of m, not constants.

Composition. With precision fixed, the OOD coverage levers apply unchanged in-distribution: k-NN-graph augmentation (self-built by the internal NN-descent pass, §7) cuts probes $\sim$3× at matched recall at 10M ($\sim$1.7× at 1M), and a dpb=4 recode ($m{=}192$: half the LUT work per candidate) trades 0.7 recall points for $\sim$1.3× QPS, net-positive on the frontier.

Effect. Cohere-768 1M, 1 thread: 0.9197@1957 QPS *graph-free* vs. HNSW 0.9122@1840 — a standalone win. At 10M/$d{=}768$, HNSW's strongest regime, the engine beats HNSW by $\sim$1.27–1.9× (1t) at matched recall once both use *self-built* graphs (Table 5); the usable graph comes from the internal NN-descent pass, not from routed self-search, which is coverage-capped at this dimension (§7).

12 Systems Techniques (Streaming)

We evaluate the streaming track on the msturing-30M-clustered runbook, targeting recall@10 under the 1 h + 8 GB budget. Three levers carry the result (Figure 11). *Batch-parallel insert* parallelizes assignment over each insert batch while preserving order, so the index is bit-identical to serial insertion. *Low-live adaptive* p grows the probe count when the live set is small, so the expected candidate count stays above threshold early in the stream. *Parallel re-rank dispatch* spreads the batched re-rank chunks across all 8 threads; serial

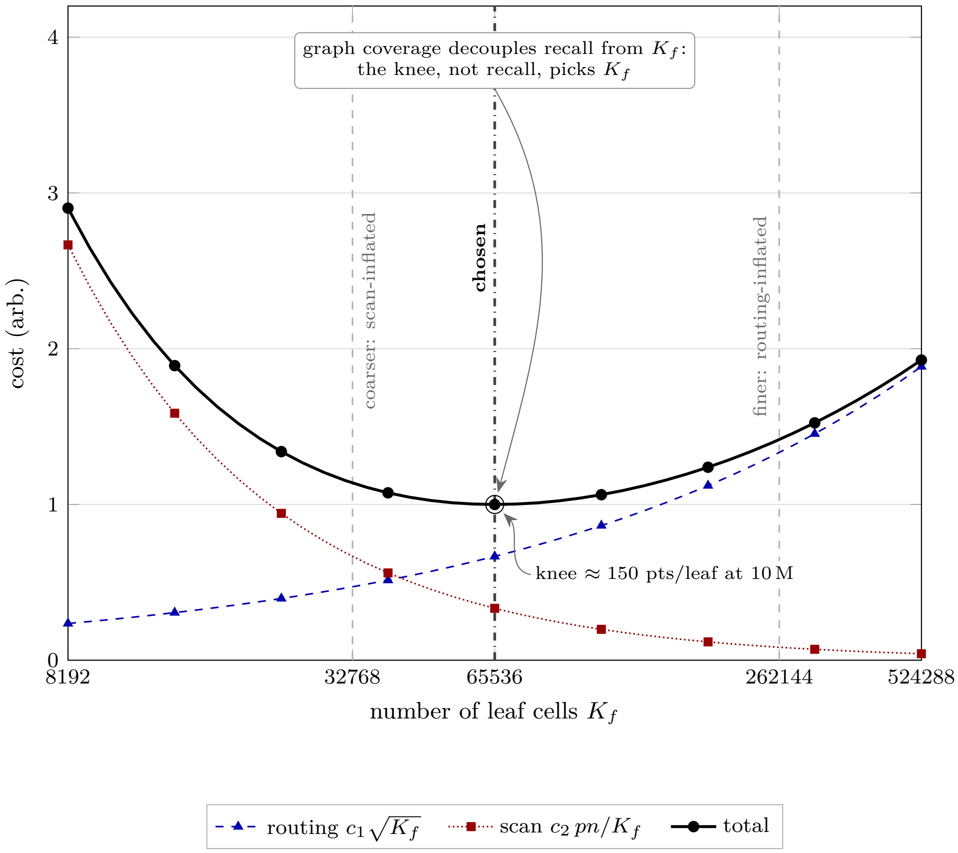


Figure 8: Analytic per-query cost model versus the number of leaf cells K_f (arbitrary units, logarithmic abscissa) for an $n = 10$ M-point index at fixed probe count p: routing cost grows as $c_1\sqrt{K_f}$, scan cost falls as $c_2\,pn/K_f$, and their sum is U-shaped. Constants are normalized so the knee of the total falls at $K_f = 65536$, i.e. roughly 150 points per leaf at 10 M. Dashed verticals mark the three built geometries: K_f=32768 (scan-inflated), the chosen K_f=65536 at the knee, and K_f=262144 (routing-inflated). Because graph coverage decouples recall from K_f, the cost knee—not a recall target—selects the leaf-cell count.

chunk dispatch never engages the Rayon thread pool and forfeits 7.6×. On top of these we add a compliance re-architecture: a first-insert-batch cold start followed by periodic live-set retraining.

Effect. Batch insert: 30M inserts 1818s → 406s (4.5×), recall-identical. Adaptive p: worst step 0.737 → 0.944. Parallel re-rank dispatch: 7.6× at 8 threads. Final compliant series (NQ=10000, 640 steps): the budget-eligible frontier on our (shared, 2–3× slower than the official VM) machine is recall@10 = 0.8821 at 52.7 min; 0.9010 needs 69 min. Recall-vs-budget is compute-bound: eligibility is machine-relative, and the same configs project to 0.90–0.92 eligible on the official 8-vCPU box. Peak memory 7.1–7.2GB < 8GB cap throughout.

13 Negative Results (Evidence for the Localization)

Each of the following was tried and *did not* pay off; we report each with its measured null or loss, since these nulls are what localize the bottleneck to routing and coverage.

- **Query-aware routing** (train cells on the query distribution): *worse* than base-trained (0.76 vs 0.89 @ matched p) — undertrained + the cells where queries are dense don't distribute base points well.
- **ADC routing** (4-bit-scan the centroids): recall-neutral but *no speed win* — the exact VNNI router is already at the compute floor.

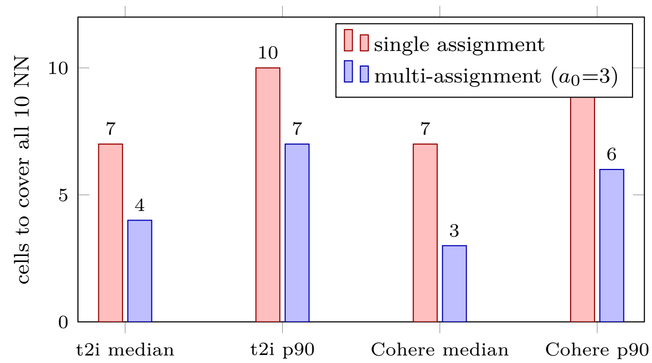


Figure 9: What multi-assignment buys, measured on the dumped assignments: the number of cells an oracle must probe to cover all 10 true neighbors drops by roughly half with a_0=3 (greedy set cover, 10M scale, both datasets). Each stored copy is a unit of *materialized* coverage — the same currency as the graph's edges (§14).

- **Exact flat routing** (score *all* finest centroids, i.e. perfect cell selection at 3× the routing cost): +0.08–0.10pt over the hierarchical descent graph-free, and +0.0pt with the graph on — the point-graph union already absorbs every descent miss. This bounds *any* routing-repair scheme (wider beams, per-level centroid graphs): the miss mass is an information cap of centroid geometry, not a descent artifact, and is recoverable only at point granularity.
- **Anisotropic/OPQ-rotated coarse codes for rank preservation**: do *not* make coarse codes rank-preserving; reorder depth is set by code bit-rate, and rotation/η move it < 0.01. Coverage-via-coarsening collapses (a scann-coverage-matched dpb=50 config yields recall 0.09).
- **RaBitQ**: no ranking edge over PQ at matched bytes.
- **Wider SIMD** (AVX-512 vpermw / 64-wide): *on OOD* d=200 the scan is memory-bound, not shuffle-bound; ≤+7%, sometimes negative (downclock/cross-lane). The *same* lever is decisive at m=384 (§11) — the negative is regime-scoped, not universal.
- **Finer cells at matched probe**: lose recall (half the probe fraction).
- **PCA/LeanVec dim reduction on OOD**: near-isotropic embeddings; dead. The same holds for *raw-basis* routing-dimension truncation in-distribution (Cohere-768: scoring a 384-dim prefix costs 1.6 recall points) — prefix truncation needs a variance-ordering rotation to be principled, in either regime.
- **Residual-encoded PQ under inner product** (baseline fairness): FAISS IVFPQ defaults `by_residual=True`, which is L_2-specific; under IP it caps text2image recall at 0.11, *decreasing* with probes. All FAISS-IVFPQ baselines here use `by_residual=False` plus exact refine — reporting the default would be a strawman.
- **Int8-prefilter + int16-rescore hybrid scan**: dominated at m=384 — the int8 pass's ordering is too noisy for any affordable keep rate (top-2000 of 8000 retains the true neighbor only 53% of the time), so the prefilter cannot shrink the int16 work without recall loss.

The OOD negatives cluster on the scan/code side and all fail there; the §6–§10 levers cluster on routing/coverage and all succeed. In-distribution the pattern inverts: the catastrophic failure is scan precision, not coverage (§11) — coverage binds again only at 10M scale, where the self-built k-NN graph supplies what routing misses (§18). Both localizations rest on the same isolation method.

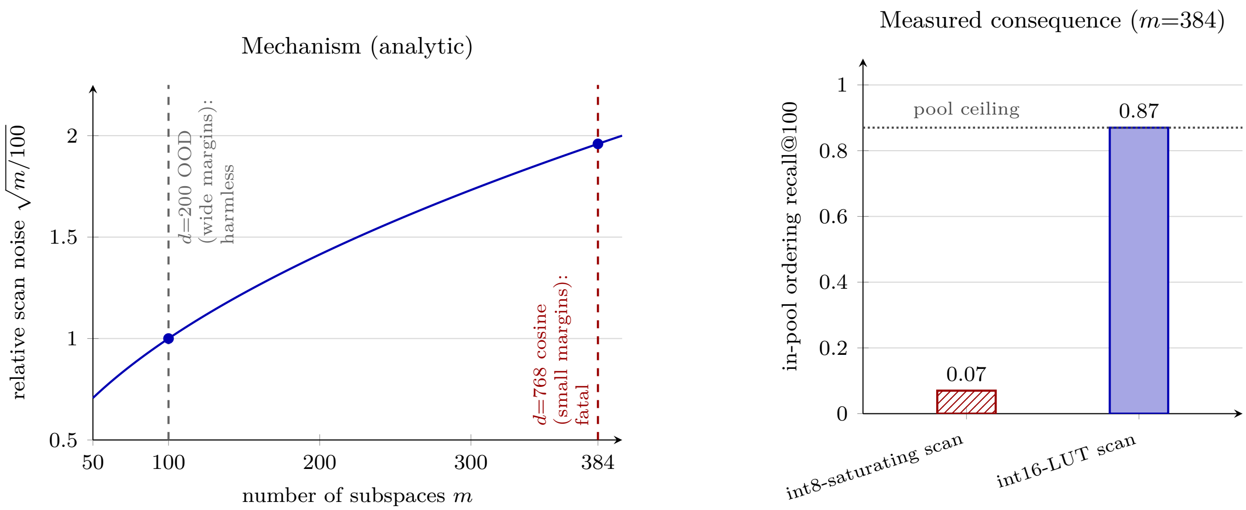


Figure 10: Scan precision must scale with code length. *Left:* an int8-saturating scan caps each per-subspace lookup at roughly 4 usable bits, so the relative quantization noise of the summed score grows as $\sqrt{m}$ with the number of subspaces m (analytic curve, normalized to 1.0 at m=100). At d=200 OOD (m=100) the wide neighbor margins absorb this noise; at d=768 cosine (m=384) the margins are small and the noise swamps them. *Right:* on identical codes at m=384, changing only the scan arithmetic from int8-saturating to an int16 LUT raises in-pool ordering recall at rank depth 100 from 0.07 to 0.87, which is exactly the pool ceiling—the scan now extracts everything the candidate pool contains.

14 Discussion: A Summary-Prediction Bottleneck

The localization result — coverage-bound OOD, precision-bound in-distribution — admits a single underlying mechanism, which an *oracle-router* analysis makes quantitative (Figure 12). Dumping the index's point→cell assignments and greedily set-covering each evaluation query's true top-10 (ground truth used only for analysis) shows the neighbors are *cell-concentrated*: a median of 3–4 cells (with a_0=3 multi-assignment) holds all ten, and an oracle that knew which cells would reach 100% coverage at p=16 on both Cohere-10M and text2image-10M. The router's geometric score achieves 0.83–0.90 at the same p: a 10–24-point gap that is *not* an occupancy problem but a *prediction* problem — which few cells hold a given query's neighbors is barely predictable from any query-independent cell summary. Three experiments bound the function class: evaluating the same score exactly (all centroids) adds $\leq$0.1pt (§13); a *learned* per-cell bias b_j — the general form of which γ is the one-parameter special case — trained on 50k held-out queries adds nothing ($-$0.8pt at p=16: the class is saturated); and a per-query adaptive probe policy triggered by a label-free signal lands on the fixed-p frontier rather than above it, because predicting per-query difficulty is the same estimation problem as routing. What crosses the bottleneck is not query-time cleverness of any kind but *materialized adjacency* — neighbor relations computed offline and stored — and the adjacency has a *direction*: base-kNN edges repair the base manifold (decisive in-distribution), while *co-retrieval* edges (base points co-occurring in a training query's top-k) reach along the query manifold and lift OOD recall 1–2 points at matched probes where base edges cannot; the two are complementary, and their hybrid sets the OOD frontier (Figure 14). Concretely: the k-NN graph (one shortlisted point reveals its true neighborhood through *precomputed edges*), and multi-assignment (each point buys a_0 cells of coverage at build time). Query-time *geometric* inference fails even when given point-level inputs: re-routing from the mean of the first-stage *results* (pseudo-relevance style) loses 2.7pt at p=16 — found points' positions do not reveal the missed cells any better than the query does — and a closed-form query transform trained to point at the true cells collapses coverage entirely ($-$40pt, regression to the mean). Result-churn feedback (correlation 0.84–0.98 with true recall gain) is the read-out side of the same structure: it *reads* the materialized edges' effect, enabling label-free drift monitoring, but cannot be converted into probe savings. The same lens explains why the seeder is load-bearing (seeds are the point-level entry: hops repair only reachable mass, and $p\leq$8 starves the corrector at both dis-

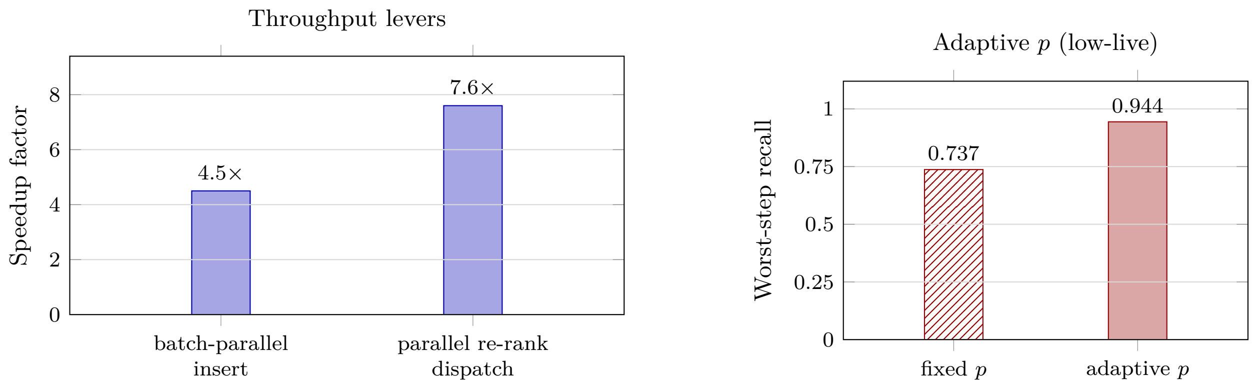


Figure 11: Streaming systems levers. Left: end-to-end speedup factors from batch-parallel insertion (30M inserts, 1818 s → 406 s; order-preserving and bit-identical to sequential insertion) and from parallel re-rank dispatch, which spreads re-rank chunks across all 8 threads (serial chunk dispatch never engages the thread pool). Right, on a separate $[0, 1]$ recall axis (not a speedup): the adaptive probe schedule for low-live phases raises worst-step recall from 0.737 (fixed p, hatched) to 0.944 (adaptive p, solid).

tributions), why scan precision must scale with code length (the $\sqrt{m}$ noise law, validated against measured margins, protects point-level ordering through the scan), and why the geometry laws (§8.2) are cost-side optima: once coverage cannot be routed into existence, geometry only balances routing against scan, and the graph decouples recall from it (§8). A practical corollary: the graph can be maintained *incrementally* — patching a 10% insert batch into a converged graph by NN-descent with per-edge aging costs 4.8× less than a rebuild and yields a better graph (0.888 vs 0.847 overlap), since converged rows keep their quality.

15 Evaluation Methodology

We report *same-hardware ratios*, never cross-machine QPS. All comparisons run on one physical machine; each competitor is measured in the same session, seconds apart, at its own best operating point.

Protocol. For a target recall (recall@10 ≥ 0.90), we (i) fix each system's operating point by an independent sweep, requiring both to clear the recall gate against the *same* exact float ground truth; (ii) alternate the two systems round to round (order flips) so neither owns a load window; (iii) take best-of-k within a round and report the *median of per-round ratios*; (iv) recompute recall from raw ids, never trusting a cached number. On a shared machine, load drift between two builds minutes apart exceeds the effects being measured, so only interleaved same-window ratios and load-independent recall are trusted.

Warm-resident semantics; pristine confirmation. Both systems measured warm and resident (leaderboard serving); cold-spawn effects are excluded by scoring each engine's warm block. Headline numbers pause all other first-party jobs (SIGSTOP) and confirm across ≥20 rounds.

The baseline at its best. At 10M, where an *official* big-ANN recipe exists, ScaNN uses it verbatim (40k leaves, anisotropic AH/LUT16, residual quant, bfloat16 re-order, top-level partitioner) with its own leaves-to-search / re-order sweep including corner configs. At scales with no official recipe (1M, 100M) we instead give ScaNN its *speed-optimal* configuration, found by sweeping the leaf count: e.g. at 1M, ScaNN peaks at 1200 leaves (6.2k QPS@0.90), with both finer and coarser partitions slower — coarser because routing over more leaves outweighs the finer scan, mirroring our own coarse-cell result (§8). This matters: a convention-chosen leaf count can silently under- or over-partition the baseline, and any win that depended on a handicapped baseline would be quarantined.

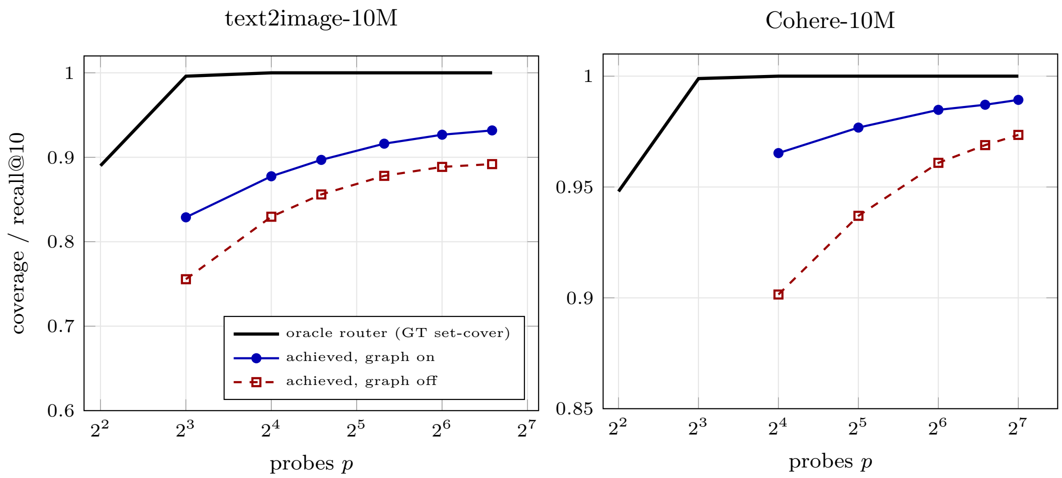


Figure 12: The summary-prediction bottleneck, quantified. An oracle router that knew which cells hold each query's true neighbors (greedy set cover over the dumped assignments, ground truth used for analysis only) reaches 100% coverage by p=16 on both datasets (black): the neighbors are cell-concentrated. The engine's calibrated geometric score (graph off, dashed) trails that roofline by 10–24 points at p=16 and closes only logarithmically with more probes — the gap is a *prediction* failure, not an occupancy one. The k-NN graph (solid blue) recovers most of the gap at every p by injecting materialized point-level adjacency (§14).

Build-time eligibility. We gate index build at **2 hours** wall-clock on this box, applied identically to both systems, so a baseline (or our own index) that cannot build in time is not merely slow — it is ineligible. This is a real constraint at 100M: a $\sqrt{n}$-scaled 126k-leaf ScaNN would take ≈32h at 8 vCPU (its partitioner k-means over the training sample), and coarser configs do not rescue it — the official 40k-leaf build was killed at 121 min and a 20k-leaf also exceeds the gate, since the O(100M) AH-encode/bf16 prep dominates regardless of leaf count — so no ScaNN configuration is eligible at 100M (Table 3). Our indices build comfortably inside the gate and *faster than ScaNN at scale*: on 16 cores, 1M in 131s, 10M in 27min, 100M in 86min, versus ScaNN's ≈ 1.5h at 10M. We therefore report build times explicitly (here, and in the note to Table 3); the hierarchical-k-means+PQ build is cheaper than ScaNN's partitioner-k-means+AH training, and the gap widens with n.

What we do not claim. Same-hardware ratios on one (shared) machine, not a standardized-VM submission; ScaNN is the *measured* opponent — gaps to binary-only leaderboard systems are *inference* from the public leaderboard.

16 Results

Figures 13 and 14 are the headline results: QPS–recall frontiers, single thread, same box, with every genbo/opponent pair from same-window sequential runs. In-distribution (Figure 13), genbo is above HNSW at every operating point shown, at all three scales — graph-free at 1M (1.5–2.4×, exceeding HNSW's recall ceiling), and with its self-built NN-descent graph at 10M (1.2–1.9×) and 35M (1.5–3.3×); HNSW plateaus (at 0.9945/0.9859/0.9818 respectively) below recall genbo reaches. Out-of-distribution (Figure 14), the picture is scale-dependent and we report it whole. At the recall-0.90 gate genbo leads everything at 10M — ScaNN at its official configuration (0.83× tight-pairwise ratio), RoarGraph at its paper configuration (1.07–1.16× ours), HNSW by ∼8.7× — and is the only build-eligible method at 100M (build 86 min; recall up to 0.9226 at 529 QPS single-thread, ∼6.7k QPS at recall 0.90 on 16 threads; ScaNN was killed at the gate, and RoarGraph's train-GT prerequisite alone projects to ∼10h from its measured 58 min at 10M). RoarGraph's structural advantage — its edges materialize *query-distribution* adjacency (bipartite projection of training queries) — is one genbo can consume directly: a *co-retrieval* graph built from the same training resource

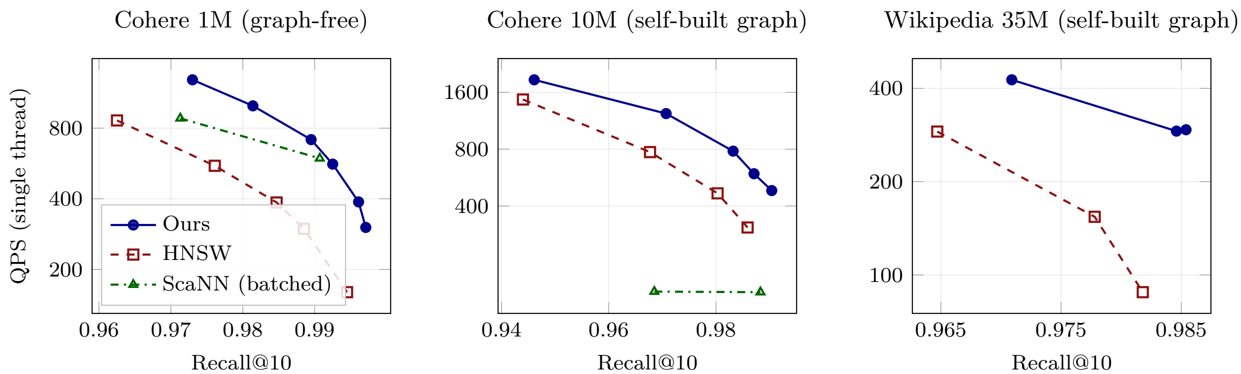


Figure 13: Recall–QPS frontiers (recall@10 versus single-thread queries per second, log scale) on Cohere 1M, Cohere 10M, and Wikipedia 35M, with both systems measured as same-window sequential pairs; our system runs graph-free at 1M and with self-built graphs at 10M and 35M. At every operating point shown, on all three scales our frontier lies above and to the right of HNSW's, i.e. higher throughput at equal recall and higher recall at equal throughput. HNSW's frontier also plateaus at recall 0.9945 (1M), 0.9859 (10M), and 0.9818 (35M), below the recall our system reaches.

Table 2: Ablation ladder at 10M (ScaNN/ours ratio; each row adds one system lever). The final configuration re-measured under the pristine protocol of §15 (other jobs paused, ≥20 rounds) gives the headline 0.83.

Configuration	ratio
γ only	1.24
+ graph coverage	1.01
+ tree-EM	0.92
+ coarse-cell geometry (knee)	0.85

(edges = co-occurrence in a training query's top-10), hybridized with the base-kNN edges, drops into the existing union mechanism with no engine change and lifts the OOD frontier 1–2 points at matched probes (§14). With it, at 1M RoarGraph keeps only the below-0.93 band (gate lead ∼1.15×, down from 1.32× against the base graph) while genbo overtakes above ≈0.97 and takes the 1M tail (0.9913 at 1533 QPS vs 0.9900 at 1353); at 10M genbo leads the entire measured range through recall 0.9945 — the best-first beam over the hybrid edges (§7) reaches 0.9927 at 382 QPS vs RoarGraph's 0.9905 at 322, with RoarGraph ahead only beyond ≈0.995; batch-mode ScaNN with deep reorder is competitive there too (mode-matched per-query it is not: 323 vs our 682 QPS at recall 0.972). genbo's OOD advantage is thus the full measured range at 10M and 100M plus build-time eligibility; the residual RoarGraph territory, after the coarse-cell corollary is applied at 1M (K_f=16384 → 2048–4096 with the hybrid graph, turning a 1.25–1.4× deficit across the band into ties-or-leads from recall 0.80 up), is the sub-0.78 sliver at 1M (∼7%) and the extreme ≥0.995 tail at 10M. Exact operating points, baseline configurations, and the streaming series are tabulated in Appendix A (Table 8). Table 2 isolates each lever's contribution at OOD-10M.

17 Related Work

Quantization-side IVF. ScaNN introduced anisotropic vector quantization (score-aware loss) and remains the measured opponent here at its official configurations; FAISS provides the reference IVF-PQ/fast-scan implementations (and §13 documents an IP-fairness pitfall in its residual default). RaBitQ represents the recent binary-ish code family; at matched bytes it showed no ranking edge in our OOD setting. Our contribution on this side is not a new code but the observation that *scan arithmetic precision must scale with code length* (§11). **Graph indices.** HNSW is the standard in-distribution baseline and is measured here at both regimes: outperformed in-distribution across the practical recall range once scan precision

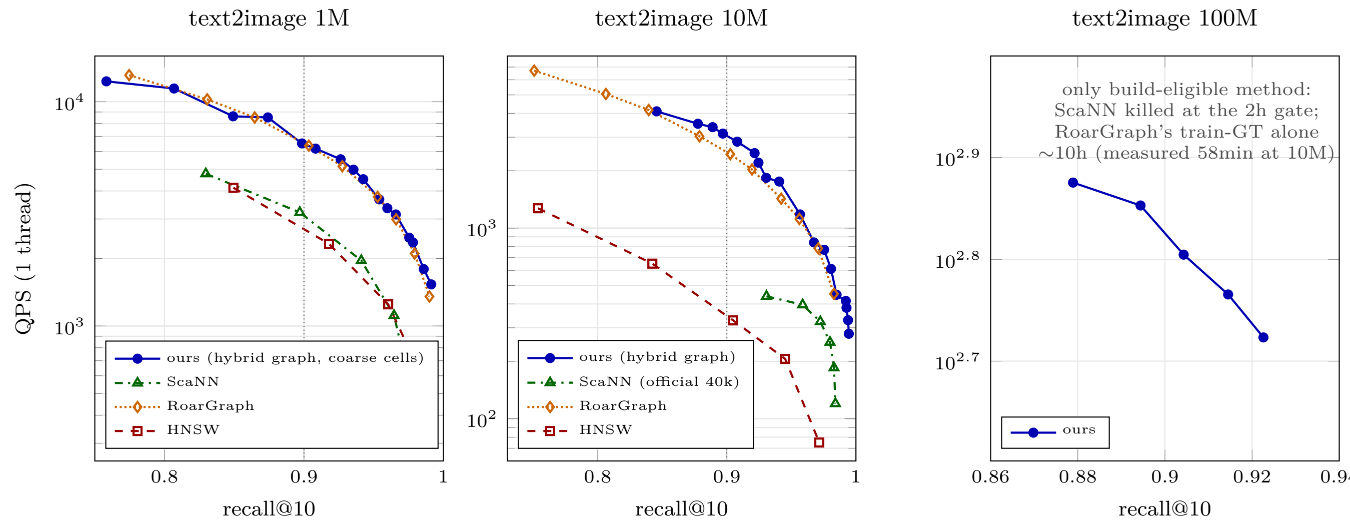


Figure 14: Out-of-distribution frontiers (text2image, d=200, inner product): QPS vs. recall@10, single thread, same box, at 1M, 10M, and 100M. Ours, ScaNN, and RoarGraph are per-query sweeps (mode-matched; ScaNN's batched API amortizes a further ∼3–4× and is then competitive with our per-query cascade above recall ≈0.96 at 10M); HNSW runs in its batched API (favorable to HNSW). Ours sweeps probe depth at hops 2–4 with the self-built k=16 graph (k=32 at the tail). Ours uses the *hybrid* graph — 16 base-kNN edges + 16 *co-retrieval* edges (bipartite projection of the track's training queries, the same resource RoarGraph consumes; §14) — as a drop-in adjacency file. At the leaderboard gate (recall 0.90, dotted), ours leads at 10M (1.25× RoarGraph per-query, 6.5× ScaNN, 8.7× HNSW), leads the entire measured 10M range through recall 0.9945 (best-first beam over the hybrid edges: 0.9927 at 382 QPS vs RoarGraph's 0.9905 at 322), and is the only eligible method at 100M; at 1M, applying the coarse-cell corollary (K_f=16384 → 2048–4096; graph coverage replaces fine partitioning, §8) closes the band: ours ties or leads RoarGraph from recall ≈0.80 upward (gate: 6.5k vs 6.4k QPS) and takes the 1M tail (0.9913 at 1533 vs 0.9900 at 1353); RoarGraph stays marginally ahead only below recall ≈0.78. A code-side IVFPQ-FastScan+refine baseline caps at recall 0.73 (10M) and does not appear.

and graph coverage are set (∼1.2–3.3× at 1M–35M, faster only below recall ≈0.95 at 35M; Tables 5, 6), and $>4\times$ behind on OOD at matched recall. RoarGraph (VLDB'24) is the OOD-specific graph method — a query-aware bipartite projection built from sampled training queries; we build it with its published hyperparameters and measure it on the same box (Table 4). DiskANN/Vamana [6] target the disk regime we do not evaluate. **OOD-specific routing.** Query-aware cell *training* fails in our hands (§13); RoarGraph solves coverage with a query-aware graph; our γ recalibration (§6) instead keeps base-trained cells and corrects the *scoring* of them, at zero query-time cost, composing with graph coverage rather than replacing the IVF skeleton. **Streaming.** The big-ANN streaming track constraints (1h wall, 8GB) shape §12; our levers there are systems-level (batch-parallel insert, adaptive probes) rather than index-structural.

18 Limitations

In-distribution at scale: a graph-build premium (self-built). At Cohere 1M/d=768 the engine wins *graph-free*. At 10M/d=768 (HNSW's strongest regime) its query path beats HNSW across the recall range (∼1.2–1.9×, higher recall throughout, and it reaches 0.99 where HNSW plateaus at ≈0.986; Table 5) — but only with an IP-k-NN graph. That graph need *not* be near-exact (Figure 15): we degraded the 0.93-overlap graph and find the win survives down to ∼0.65 overlap (genbo still beats HNSW on both recall and QPS, at a modestly higher probe count), with the recall benefit saturating quickly — even a 0.28-overlap graph recovers most of it. Crucially, the graph requires no external builder: genbo's *routed* self-search is coverage-capped at this dimension (it reaches only ∼0.53 exact-overlap@16, and even probing 6% of cells lifts this to

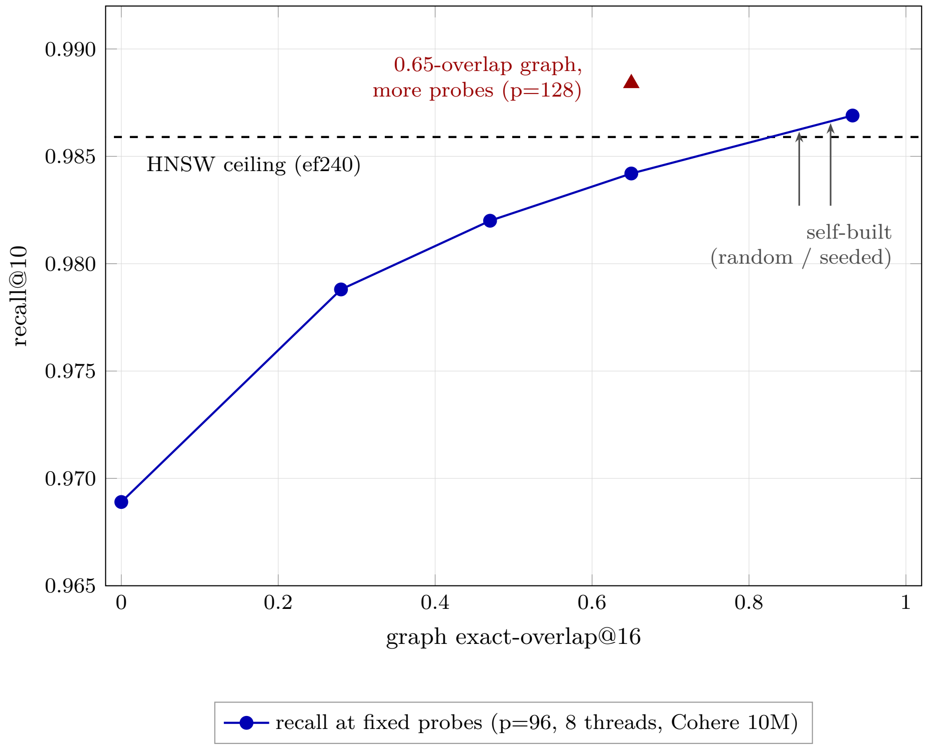


Figure 15: Recall@10 on Cohere 10M at a fixed probe budget (p=96, 8 threads) as a function of the exact-overlap@16 of the neighborhood graph; the leftmost point (overlap 0) runs with no graph at all. The recall benefit saturates rapidly: most of the gain is already realized at 0.28 overlap. A 0.65-overlap graph combined with a modestly higher probe count (p=128, triangle) already clears the HNSW ceiling (ef240, dashed), so a near-exact graph only sharpens the margin. Arrows mark the overlap achieved by self-built graphs with random and seeded initialization.

just $\sim$0.63 — the true neighbours are scattered across too many Voronoi cells for any practical probe count to cover), but its internal *NN-descent* pass (§7) does not route at all and self-builds a 0.904-overlap graph in minutes at 10M — query-time performance equal-or-better than the multi-hour external near-exact graph. What remains is honest accounting of the *build-time premium*: the descent pass adds minutes to the build (random init 24 min for 0.864 overlap; 8 min of descent on a 35-min routed self-kNN pass for 0.904 — either way under HNSW's own 48-min 10M build on the same box), a cost graph methods like DiskANN/NSG already assume as their core build step. The reason genbo needs the graph at all is a *candidate-efficiency* gap we profiled: graph-free, to reach recall 0.96 it scans and exactly-rescores $\sim$1600 candidates/query (memory-latency-bound $\sim$0.6 μs/row, already prefetched) where HNSW's greedy traversal touches fewer — the graph supplies exactly the coverage the routing misses. The same recipe — self-built graph plus multi-hop repair (§7) — carries to higher dimension and scale: at 35M/d=1024 (Wikipedia Cohere-v3, unit-norm, strongly anisotropic $\|\mu\|$=0.48) genbo *beats* a memory-matched HNSW across the practical high-recall range using its own NN-descent graph (1.5–3.3× at recall $\geq$0.96, reaching recall 0.985 HNSW cannot; Table 6), HNSW faster only below $\approx$ 0.95 — there HNSW is weaker (it plateaus near 0.982) and genbo's self-built graph matches a competitor-built one (0.965 vs 0.970 overlap@16). **Benchmark hygiene (a cautionary note).** A ground-truth construction pitfall can silently reverse a comparison on this kind of carved-query dataset. When the query set is carved from the base and the exact ground truth is computed over the base *including* those rows, an index that correctly *excludes* the queries faces an artificial recall ceiling — on this dataset 38% of each top-10 are query self-matches or near-duplicate query rows, capping achievable recall at 0.62 — while an index that inadvertently *retains* the queries appears to win. All numbers here use ground truth recomputed over the queries-excluded base, with every method searching that same base. The failure mode is silent: both indices run without error and the ceiling looks like a model deficiency. **Other scope:** single (shared) machine, not a standardized-VM submission (the gating experiment for a leaderboard claim); γ and scan-precision are fit/selected per dataset; the coarse-cell corollary is shown on IP/OOD data; the gap to binary-only leaderboard systems is inference from public numbers; filter/sparse tracks are untouched.

Table 3: OOD text2image, recall@10-matched QPS ratio (ScaNN/ours; < 1 = our engine faster). Same-hardware, tight-pairwise; ScaNN at its official config at 10M and its swept speed-optimal config (1200 leaves) at 1M (§15). At 100M no ScaNN index builds within the gate (note), so the comparison is on build-eligibility.

Scale	1-thread	8-thread	recall@10 gate
1M	0.755	0.746	0.905
10M	0.827	0.758	0.901
100M		ScaNN build-ineligible (> 2h gate; see note)	

At 100M no ScaNN configuration builds within the 2h gate (40k-leaf killed at 121 min; a coarser 20k-leaf also exceeds 2h — the O(100M) AH-encode/bf16 prep dominates regardless of leaf count), so no matched-recall ratio exists. Ours builds a 0.903-recall index in 86 min and serves ∼6.7k QPS at recall 0.90 (16 threads) — under the 2h gate it is the only method with a 100M index.

References

[1] R. Guo et al. Accelerating large-scale inference with anisotropic vector quantization. *ICML*, 2020.

[2] P. Sun, R. Guo, S. Kumar. SOAR: improved indexing for approximate nearest neighbor search. *NeurIPS*, 2023.

[3] Yu. Malkov, D. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *TPAMI*, 2020.

[4] M. Douze et al. The Faiss library. *arXiv:2401.08281*, 2024.

[5] M. Chen et al. RoarGraph: a projected bipartite graph for efficient cross-modal approximate nearest neighbor search. *PVLDB*, 2024.

[6] S. Jayaram Subramanya et al. DiskANN: fast accurate billion-point nearest neighbor search on a single node. *NeurIPS*, 2019.

[7] W. Dong, M. Charikar, K. Li. Efficient k-nearest neighbor graph construction for generic similarity measures. *WWW*, 2011.

[8] J. Gao, C. Long. RaBitQ: quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *SIGMOD*, 2024.

[9] H. Simhadri et al. Results of the big-ANN: NeurIPS'23 competition. *arXiv:2409.17424*, 2024.

[10] F. André, A.-M. Kermarrec, N. Le Scouarnec. Quicker ADC: unlocking the hidden potential of product quantization with SIMD. *TPAMI*, 2021.

A Full Operating Points and Baseline Details

Exact measured operating points behind Figures 13, 14, and 7, with per-baseline configuration notes, the geometry table, and the streaming table.

Table 4: Multi-system baselines, OOD text2image-10M, 1 thread, same box, same exact ground truth. QPS at the listed recall@10 (each system swept to its knee; FAISS-IVFPQ uses `by_residual=False` + exact refine, its fair IP configuration). RoarGraph [5] is the published OOD-specific graph method, built with its paper hyperparameters (M_{sq}=100, M=35, L=500, 2M sampled train queries, GT recall@100 0.96).

System	recall@10	QPS (1t)	note
ours	0.905	7657 (1M) / 2804 (10M)	IVF-PQ cascade
ScaNN (official 40k)	0.901	(ratio 0.827 vs ours)	IVF-AH
RoarGraph	0.903	2250‡	OOD bipartite graph
HNSW (faiss, M32)	0.905	328	needs ef≥160 on OOD
IVFPQ-FastScan+refine	cannot reach 0.90 (caps 0.73)		code-side fails OOD

‡5-round same-window pairwise median vs ours at matched recall: ours/RoarGraph = 1.07×.

Table 5: In-distribution Cohere-768 (cosine, unit-norm), 1 thread, same box, same exact ground truth. On the shared machine only interleaved same-window ratios are trusted (§15); the 10M rows are clean-window sequential pairs (each genbo/HNSW pair run back-to-back on the same cores). Recall is load-independent.

Scale	System	recall@10	QPS (1t)
1M	**ours** (int16, graph-free)	**0.9197**	**1957**
1M	HNSW (ef40)	0.9122	1840
1M	ScaNN (2k leaves, reorder 600)	0.9147	1306
1M	IVFPQ+refine (nprobe 256)	0.9014	877
1M	**ours** (graph-free, p=192 / p=384; top)	**0.9895 / 0.9961**	**715 / 388**
1M	HNSW (ef240 / ef640; its ceiling)	0.9847 / 0.9945	386 / 160
10M	**ours** (*self-built* graph, hops=2/3)	**0.9707 / 0.9832**	**1234 / 781**
10M	HNSW (ef96 / ef160)	0.9677 / 0.9803	772 / 467
10M	**ours** (*self-built* graph, hops=3, p=128; top)	**0.9904**	483
10M	HNSW (ef240; its ceiling)	0.9859	308
10M	ours (*routed* self-search graph, hops=3)	0.9619	751
10M	ScaNN (8k leaves, deep reorder)	0.9461	∼330

At **1M** the engine wins *graph-free* and standalone at every operating point through HNSW's ceiling (1.5× at recall 0.97 rising to 2.4× at HNSW's 0.9945 ceiling, which it exceeds — 0.9971 at 1.9× the QPS; a 66-second NN-descent graph adds a further ∼1.5×, e.g. 0.9936 at 736 vs ef640's 160). At **10M/d**=768 (HNSW's strongest regime) the engine beats HNSW at every operating point using its *self-built* NN-descent graph (clean-window sequential runs, same cores): ∼1.27× at recall 0.946, ∼1.6× at 0.97, ∼1.67× at 0.983, ∼1.9× at HNSW's 0.9859 ceiling (1 thread; 1.2–1.6× at 8 threads), higher recall throughout, and it reaches recall 0.99 where HNSW plateaus (≈ 0.986). The graph *need not be near-exact*: degrading its overlap@16 from 0.93 to 0.65 costs only ∼0.3pt recall at matched probes, and genbo still beats HNSW on *both* recall and QPS with a 0.65-overlap graph (at a modestly higher probe count); even a 0.28-overlap graph recovers most of the coverage benefit. QPS is nearly flat across graph quality — the graph buys coverage/recall, not speed. And the graph is *self-built*: the engine's internal NN-descent pass (§7) reaches 0.864 overlap in 24 min standalone at 10M (0.904 warm-started; either under HNSW's own 48-min build), and the resulting self-built-graph rows are query-time equal-or-better than the external near-exact graph. (The *routed* self-search variant, last row, is the one that fails — it is coverage-capped at d=768 and loses to HNSW; NN-descent does not route, which is the point.) ScaNN trails throughout.

Table 6: In-distribution Wikipedia Cohere-v3 (35M, d=1024, unit-norm, strongly anisotropic $\|\mu\|$=0.48), 1 thread, same box. Ground truth is recomputed over the queries-excluded base and HNSW is rebuilt on that same base (int8 scalar-quantized storage), so both systems search identical data (§18). Recall is load-independent.

Recall band	System	recall@10	QPS (1t)
∼0.97	**ours** (dpb4 + self graph hops=2, p=48)	**0.9709**	**426**
∼0.97	HNSW (ef160)	0.9647	290
∼0.98	**ours** (dpb4 + self graph hops=2, p=96)	**0.9846**	**291**
∼0.98	HNSW (ef320)	0.9778	154
top	**ours** (hops=3, p=96)	**0.9854**	**294**
top	HNSW (ef640, its ceiling)	0.9818	88

The graph here is *self-built* by the engine's internal NN-descent pass (§7; 0.87 → 0.965 overlap@16 in 9.5 min, matching the 0.970 of an HNSW-built graph) — nothing is borrowed from a competitor; the comparison is self-contained. Same-window sequential 1-thread runs: ours is ∼1.5× faster at recall 0.97 (and +0.6pt), ∼1.9× at 0.98, ∼3.3× at HNSW's ceiling — and reaches recall 0.985 that HNSW cannot; the win holds at 8 threads. HNSW is faster only below recall ≈0.95.

Table 7: Tree geometry and per-query routing cost (Eq. 2) by scale. Beams are aggressive ($b_0 \approx$ 3–25% of coarse cells) because OOD queries route poorly. The 100M geometry is validated by a direct fine-vs-coarse K_f A/B at 100M, not read off the 2-level scaling law (§8.2); the cost model (§8.1) motivates its third level.

scale	L	K_f	cell counts	beams	route evals	pts/leaf
1M	2	16,384	C_0=768	b_0=96	$768 + 2{,}048 = 2{,}816$	61
10M	2	65,536	C_0=4096	b_0=128	$4096 + 128{\cdot}16 = 6{,}144$	152
100M	3	524,288	C_0=2048, C_1=32768	b_0=512, b_1=256	$2048 + 512{\cdot}16 + 256{\cdot}16 = 14{,}336$	190

Table 8: Streaming (msturing-30M-clustered runbook; NQ=10000, 640 steps) on our shared box under the 8 GB DRAM + 1 h budget. Recall-vs-budget is compute-bound and machine-relative — this box runs ∼2–3× slower than the official 8-vCPU VM, where the same configs project to 0.90–0.92 eligible.

Operating point	recall@10	wall (min)	peak mem
budget-eligible frontier (this box)	0.8821	52.7	7.1–7.2 GB
higher recall (over 1 h here)	0.9010	69.0	7.1–7.2 GB

Enabling levers (§12): batch-parallel insert (30M inserts 1818 → 406 s, 4.5×, recall-identical); low-live adaptive p (worst step 0.737 → 0.944); parallel re-rank dispatch (7.6× at 8 threads). Both points stay under the 8 GB cap.